HADOOP

Travaux pratiques : installer, stocker, ordonnancer et calculer

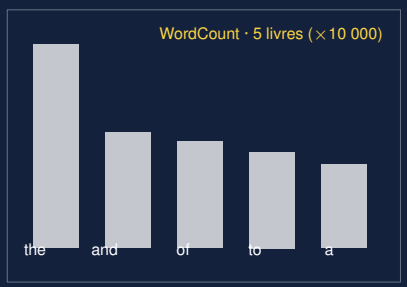
Auteur : **H. BADIR AI**

0 TP-0 Installer et configurer Hadoop (Windows ou Docker)

1 TP-1 HDFS : architecture et commandes

2 TP-2 YARN : ressources et applications

3 TP-3 MapReduce pas à pas : mots et amis communs



Sommaire

TP

H. BADIR AI • Big Data — Écosystème Hadoop

0 Installer et configurer Hadoop 3
0.1 Option A — Hadoop directement sous Windows 4
0.2 Option B — Hadoop dans Docker. 6
0.3 Vérifier l'installation (les deux options) 8

1 HDFS : architecture et commandes 11
1.1 Rappel : l'architecture de HDFS 11
1.2 Les commandes essentielles 13
1.3 Pas à pas : cinq livres dans HDFS 13

2 YARN : ressources et applications 17
2.1 Rappel : l'architecture de YARN 17
2.2 Les commandes essentielles 18
2.3 Pas à pas : lancer, suivre, arrêter 19
2.4 Pas à pas : une file d'attente « dev » 20

3 MapReduce pas à pas : mots et amis communs 23
3.1 Rappel : le modèle MapReduce 23
3.2 Préparation : le projet Java 24
3.3 Cas 1 — Compter les mots dans 5 livres 25
3.4 Cas 2 — Les amis communs (2, 3 puis 4 personnes) 29

4 Références 38

0 Installer et configurer Hadoop

H. BADIR AI • Big Data — Écosystème Hadoop

Dans ce TP, vous installez un Hadoop **complet** (HDFS + YARN + MapReduce), dans la version stable **3.4.3**, de l'une des deux façons suivantes :

1. **Choisir : Windows natif ou Docker**
2. **Installer Java (JDK 11) et Hadoop**
3. **Configurer les 4 fichiers XML**
4. **Formater HDFS et démarrer les démons**
5. **Vérifier : jps, ports 9870 et 8088, job « pi »**

Le saviez-vous ?

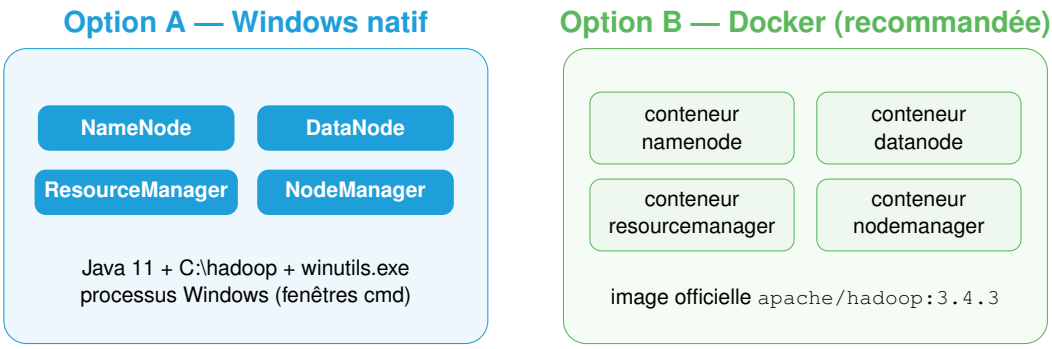
La branche 3.4 est la ligne stable la plus utilisée en 2026 : la 3.4.3 date de février 2026. La 3.5.0 (avril 2026) existe, mais elle est plus récente et encore peu déployée. Pour un TP, on privilégie la **stabilité**.

Rappel 0.1 — Ce que l'on installe

Hadoop n'est pas un seul programme : c'est un ensemble de **démons** Java qui tournent en permanence.

Démon	Composant	Rôle
NameNode	HDFS (maître)	Connaît l'arborescence et l'emplacement de chaque bloc
DataNode	HDFS (esclave)	Stocke les blocs sur son disque
ResourceManager	YARN (maître)	Distribue la mémoire et le CPU du cluster
NodeManager	YARN (esclave)	Lance les containers qui exécutent les tâches

En mode **pseudo-distribué**, les 4 démons tournent sur la même machine : c'est la configuration de ce TP.



✓ Option A — Windows natif
Aucun logiciel tiers ; on voit les vrais fichiers de configuration ; idéal pour comprendre.
Mais : il faut `winutils.exe` (non officiel) ; les chemins avec espaces posent problème ; un seul nœud.

✓ Option B — Docker
Identique sur Windows, macOS et Linux ; plusieurs DataNodes possibles ; on repart de zéro en une commande.
Mais : il faut 8 Go de RAM et Docker Desktop (WSL 2 sous Windows).

0.1 Option A — Hadoop directement sous Windows

✍ Attention ! — les chemins

Hadoop sous Windows supporte mal les **espaces** dans les chemins (`C:\Program Files\...`). Installez Java dans `C:\Java` et Hadoop dans `C:\hadoop`. Ouvrez toujours l'invite de commandes **en tant qu'administrateur**.

1 Installer Java 11

Téléchargez le JDK 11 **Eclipse Temurin** (fichier .msi, adoptium.net). Pendant l'installation, choisissez le dossier `C:\Java\jdk-11`. Hadoop 3.4 fonctionne avec Java 8 ou 11 : on prend Java 11.

2 Télécharger et décompresser Hadoop 3.4.3

PowerShell (administrateur)

```
> cd $env:USERPROFILE\Downloads
> curl.exe -L -O https://dlcdn.apache.org/hadoop/common/hadoop-3.4.3/hadoop
    -3.4.3.tar.gz
> tar -xzf hadoop-3.4.3.tar.gz -C C:\
> Rename-Item C:\hadoop-3.4.3 C:\hadoop
# si dlcdn ne répond plus : https://archive.apache.org/dist/hadoop/common/hadoop
    -3.4.3/
```

3 Ajouter winutils.exe et hadoop.dll

Hadoop a besoin de deux binaires natifs Windows que la fondation Apache ne distribue pas. On utilise ceux compilés pour la 3.4 (dépôt GitHub `kontext-tech/winutils`, dossier `hadoop-3.4.0-win10-x64/bin`). Copiez `winutils.exe` et `hadoop.dll` dans `C:\hadoop\bin`.

> ✍ **Attention !**
> Ces fichiers sont fournis par la communauté « pour l'apprentissage » : ne les utilisez pas en production. La version 3.4.0 fonctionne avec les autres versions 3.4.x.

4 Variables d'environnement

PowerShell (administrateur)

```
> setx JAVA_HOME "C:\Java\jdk-11" /M
> setx HADOOP_HOME "C:\hadoop" /M
# Puis : Paramètres > Système > Variables d'environnement > Path > Nouveau :
#    %JAVA_HOME%\bin     %HADOOP_HOME%\bin     %HADOOP_HOME%\sbin
```

Fermez et rouvrez l'invite de commandes, puis vérifiez avec `java -version` et `hadoop version`. Dans `C:\hadoop\etc\hadoop\hadoop-env.cmd`, remplacez la ligne `set JAVA_HOME=...` par `set JAVA_HOME=C:\Java\jdk-11`.

5 Configurer les 4 fichiers XML

Les quatre fichiers sont dans `C:\hadoop\etc\hadoop`. Remplacez la balise `<configuration/>` de chacun :

core-site.xml -- adresse de HDFS

```
<configuration>
  <property><name>fs.defaultFS</name><value>hdfs://localhost:9000</value></
    property>
</configuration>
```

hdfs-site.xml -- réplication et dossiers de stockage

```
<configuration>
  <property><name>dfs.replication</name><value>1</value></property>
  <property><name>dfs.namenode.name.dir</name><value>file:///C:/hadoop/data/
    namenode</value></property>
  <property><name>dfs.datanode.data.dir</name><value>file:///C:/hadoop/data/
    datanode</value></property>
</configuration>
```

mapred-site.xml -- MapReduce s'exécute sur YARN

```
<configuration>
  <property><name>mapreduce.framework.name</name><value>yarn</value></property>
  <property><name>mapreduce.application.classpath</name>
    <value>%HADOOP_HOME%/share/hadoop/mapreduce/*,%HADOOP_HOME%/share/hadoop/
    mapreduce/lib/*</value></property>
</configuration>
```

yarn-site.xml -- service de shuffle et variables transmises

```
<configuration>
  <property><name>yarn.nodemanager.aux-services</name><value>mapreduce_shuffle</
    value></property>
  <property><name>yarn.nodemanager.env-whitelist</name>
    <value>JAVA_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,HADOOP_CONF_DIR,
    CLASSPATH_PREPEND_DISTCACHE,HADOOP_YARN_HOME,HADOOP_MAPRED_HOME</value></
    property>
  <property><name>yarn.log-aggregation-enable</name><value>true</value></property
    >
</configuration>
```

6 Formater HDFS et démarrer

Invite de commandes (administrateur)

```
> mkdir C:\hadoop\data\namenode C:\hadoop\data\datanode
> hdfs namenode -format
# ... INFO common.Storage: Storage directory C:\hadoop\data\namenode has been
    successfully formatted.
> start-dfs.cmd
> start-yarn.cmd
> jps
4112 NameNode
5220 DataNode
6304 ResourceManager
7088 NodeManager
7712 Jps
```

`start-dfs.cmd` et `start-yarn.cmd` ouvrent chacun deux fenêtres : une par démon. Ne les fermez pas. Pour tout arrêter : `stop-yarn.cmd` puis `stop-dfs.cmd`.

✍ Attention ! — à ne faire qu'une fois

`hdfs namenode -format` **efface** les métadonnées de HDFS. Si vous le relancez, le DataNode refuse ensuite de démarrer (« Incompatible clusterIDs »). Il faut alors vider `C:\hadoop\data\datanode`.

0.2 Option B — Hadoop dans Docker

1 Installer Docker

Windows 10/11 : activez WSL 2 (`wsl --install` dans PowerShell administrateur, puis redémarrez) et installez **Docker Desktop**. macOS : Docker Desktop. Linux : `curl -fsSL https://get.docker.com | sudo sh`. Vérifiez avec `docker compose version`.

2 Décrire le cluster

Créez un dossier `hadoop-docker` contenant les deux fichiers suivants, ainsi que deux sous-dossiers vides, `jobs` et `donnees`, partagés avec le conteneur.

docker-compose.yml

```yaml
services:
  namenode:
    image: apache/hadoop:3.4.3
    hostname: namenode
    command: ["hdfs", "namenode"]
    ports: ["9870:9870"]
    env_file: ./config
    environment:
      ENSURE_NAMENODE_DIR: "/tmp/hadoop/dfs/name"   # formate HDFS au 1er dé
    marrage
    volumes: ["./jobs:/opt/jobs", "./donnees:/opt/donnees"]
  datanode:
    image: apache/hadoop:3.4.3
    command: ["hdfs", "datanode"]
    env_file: ./config
  resourcemanager:
    image: apache/hadoop:3.4.3
    hostname: resourcemanager
    command: ["yarn", "resourcemanager"]
    ports: ["8088:8088"]
    env_file: ./config
  nodemanager:
    image: apache/hadoop:3.4.3
    command: ["yarn", "nodemanager"]
    env_file: ./config
```

config -- chaque ligne devient une propriété XML

```
CORE-SITE.XML_fs.defaultFS=hdfs://namenode:8020
HDFS-SITE.XML_dfs.namenode.rpc-address=namenode:8020
HDFS-SITE.XML_dfs.namenode.name.dir=file:///tmp/hadoop/dfs/name
HDFS-SITE.XML_dfs.datanode.data.dir=file:///tmp/hadoop/dfs/data
HDFS-SITE.XML_dfs.replication=2
HDFS-SITE.XML_dfs.permissions.enabled=false
HDFS-SITE.XML_dfs.namenode.datanode.registration.ip-hostname-check=false
MAPRED-SITE.XML_mapreduce.framework.name=yarn
MAPRED-SITE.XML_yarn.app.mapreduce.am.env=HADOOP_MAPRED_HOME=/opt/hadoop
MAPRED-SITE.XML_mapreduce.map.env=HADOOP_MAPRED_HOME=/opt/hadoop
MAPRED-SITE.XML_mapreduce.reduce.env=HADOOP_MAPRED_HOME=/opt/hadoop
MAPRED-SITE.XML_mapreduce.map.memory.mb=1024
MAPRED-SITE.XML_mapreduce.reduce.memory.mb=1024
```

```
MAPRED-SITE.XML_yarn.app.mapreduce.am.resource.mb=1024
YARN-SITE.XML_yarn.resourcemanager.hostname=resourcemanager
YARN-SITE.XML_yarn.nodemanager.aux-services=mapreduce_shuffle
YARN-SITE.XML_yarn.nodemanager.resource.memory-mb=4096
YARN-SITE.XML_yarn.scheduler.minimum-allocation-mb=512
YARN-SITE.XML_yarn.nodemanager.pmem-check-enabled=false
YARN-SITE.XML_yarn.nodemanager.vmem-check-enabled=false
YARN-SITE.XML_yarn.log-aggregation-enable=true
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.queues=default
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.default.capacity=100
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.maximum-am-resource-percent=0.5
```

3 Démarrer et entrer dans le cluster

Terminal du PC

```
$ cd hadoop-docker
$ docker compose up -d --scale datanode=2 --scale nodemanager=2
$ docker compose ps                    # 6 conteneurs « running »
$ docker compose exec namenode bash    # shell DANS le cluster (invite bash-...$)
```

Toutes les commandes `hdfs`, `yarn` et `hadoop` des TP suivants se tapent **dans ce shell**. Pour arrêter : `docker compose stop`, qui conserve les données. `docker compose down`, en revanche, **supprime** les conteneurs et donc le contenu de HDFS.

☞ Astuce — peu de RAM ?

Avec 8 Go, démarrez une version allégée : `docker compose up -d` (1 DataNode, 1 NodeManager) et passez `dfs.replication` à 1.

0.3 Vérifier l'installation (les deux options)

Rappel 0.2 — La liste de contrôle

1. **Interfaces web** : HDFS sur `http://localhost:9870` (onglet *Datanodes*), YARN sur `http://localhost:8088` (onglet *Nodes*).
2. **Premier fichier** dans HDFS :

Shell du cluster (ou cmd Windows)

```
$ hdfs dfs -mkdir -p /user/etudiant
$ echo "Bonjour Hadoop" > bonjour.txt
$ hdfs dfs -put bonjour.txt /user/etudiant/
$ hdfs dfs -cat /user/etudiant/bonjour.txt
Bonjour Hadoop
```

3. **Premier job MapReduce** : le calcul de π fourni avec Hadoop.

Shell du cluster

```
$ hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples
    -3.4.3.jar pi 10 100
Job Finished in 31.4 seconds
Estimated value of Pi is 3.14800000000000000000
```

Sous Windows, écrivez `%HADOOP_HOME%` au lieu de `$HADOOP_HOME`.

★ À retenir

Une installation est validée quand : **(1)** les 4 démons tournent, **(2)** les ports 9870 et 8088 répondent, **(3)** un fichier peut être écrit puis relu dans HDFS, **(4)** le job `pi` se termine avec « Job Finished ».

Symptôme	Cause probable	Remède
`JAVA_HOME is incorrectly set`	espace dans le chemin de Java	JDK dans `C:\Java`, modifier hadoop-env.cmd
`Could not locate winutils.exe`	winutils absent de `bin`	copier winutils.exe et hadoop.dll
DataNode absent de `jps`	HDFS formaté deux fois	vider le dossier datanode, redémarrer
Job bloqué en `ACCEPTED`	pas assez de mémoire pour l'AM	`maximum-am-resource-percent`, RAM
`Connection refused :8032`	ResourceManager arrêté	`start-yarn.cmd` / `docker compose ps`

Exercices du TP-0

0.1 — *Versions*

Donnez les commandes qui affichent la version de Java et celle de Hadoop. Quelle version de Java la branche Hadoop 3.4 accepte-t-elle à l'exécution ?

0.2 — *Les démons*

Lancez `jps` (Windows), ou `docker compose ps` (Docker). Associez chaque processus à son composant (HDFS ou YARN) et à son rôle (maître ou esclave).

0.3 — *Lire la configuration*

Sans ouvrir les fichiers XML, affichez la valeur de `dfs.replication` et de `fs.defaultFS`. Expliquez la différence entre les deux options d'installation.

0.4 — *Le job pi*

Lancez `pi 4 1000`. Combien de tâches Map et Reduce sont lancées ? Combien de containers YARN au minimum ? La précision s'améliore-t-elle avec `pi 4 100000` ?

0.5 — *Persistance (Docker)*

Écrivez un fichier dans HDFS, puis faites `docker compose stop` et `docker compose start`. Le fichier existe-t-il toujours ? Et après `docker compose down` puis `up -d` ?

Corrigé express

0.1 `java -version`, `hadoop version`. Hadoop 3.4 s'exécute sur Java 8 ou 11.
0.2 NameNode (HDFS, maître), DataNode (HDFS, esclave), ResourceManager (YARN, maître), NodeManager (YARN, esclave) ; Jps est l'outil lui-même.
0.3 `hdfs getconf -confKey dfs.replication` → 1 (Windows) ou 2 (Docker) ; `hdfs getconf -confKey fs.defaultFS` → `hdfs://localhost:9000` (Windows) ou `hdfs://namenode:8020` (Docker).
0.4 4 Map + 1 Reduce, plus 1 ApplicationMaster, soit 6 containers (pas forcément simultanés). Oui : plus d'échantillons, donc une meilleure estimation (méthode quasi-Monte-Carlo, à base de suites de Halton : le résultat est déterministe).
0.5 Après stop/start : **oui**, les conteneurs sont conservés. Après down/up : **non**, les conteneurs (et leurs disques) sont recréés et HDFS est reformaté.

1 HDFS : architecture et commandes

H. BADIR AI • Big Data — Écosystème Hadoop

HDFS (*Hadoop Distributed File System*) stocke de très gros fichiers en les découpant en **blocs** copiés sur plusieurs machines. Dans ce TP :

- **Rappel : NameNode, DataNodes, blocs**
- **Les commandes `hdfs dfs`**
- **Administration : `fsck`, `dfsadmin`**
- **Pas à pas avec 5 livres**
- **10 exercices corrigés**

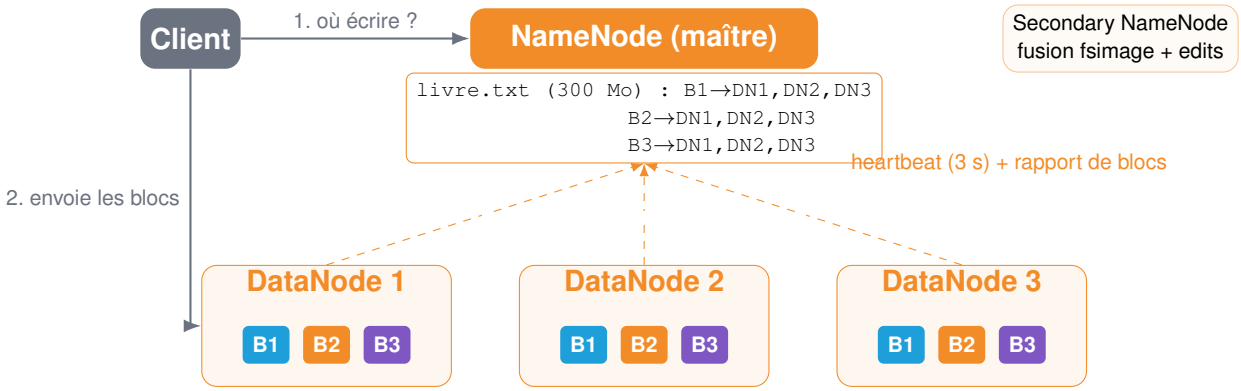
Le saviez-vous ?

HDFS est inspiré du **Google File System** (2003). Un bloc fait **128 Mo** par défaut, contre 4 Ko sur un disque de PC : on lit peu de gros morceaux plutôt que beaucoup de petits.

1.1 Rappel : l'architecture de HDFS

Définition — Bloc, réplication, NameNode et DataNode

Un fichier est découpé en **blocs** de taille fixe (`dfs.blocksize`, 128 Mo). Chaque bloc est copié **r** fois (`dfs.replication`, 3 par défaut) sur des **DataNodes** différents. Le **NameNode** garde en mémoire l'arborescence et la liste des blocs de chaque fichier, mais **aucune donnée**.



Rappel 1.1 — Le calcul à savoir faire

Un fichier de taille T, stocké avec des blocs de taille B et une réplication r, occupe :

$$\text{Nombre de blocs} = \lceil T/B \rceil \qquad \text{Espace disque brut} = T \times r$$

(le dernier bloc n'occupe que sa taille réelle : il n'est jamais complété)

1. Un fichier de 300 Mo avec des blocs de 128 Mo donne $\lceil 300/128 \rceil = 3$ blocs : 128 + 128 + 44 Mo.
2. Avec $r = 3$, il occupe $3 \times 300 = 900$ Mo sur l'ensemble des disques du cluster, et non $3 \times 3 \times 128$.

1.2 Les commandes essentielles

Commande	Effet
`hdfs dfs -ls [-R] [-h] <chemin>`	lister (2e colonne = réplication)
`hdfs dfs -mkdir -p <rép>`	créer un répertoire (et ses parents)
`hdfs dfs -put <local> <hdfs>`	copier du PC vers HDFS (`-f` pour écraser)
`hdfs dfs -get <hdfs> <local>`	copier de HDFS vers le PC
`hdfs dfs -cat | -head | -tail <f>`	afficher tout, le premier Ko ou le dernier Ko
`hdfs dfs -cp | -mv <src> <dst>`	copier, déplacer ou renommer dans HDFS
`hdfs dfs -rm [-r] [-skipTrash] <ch>`	supprimer un fichier ou un répertoire
`hdfs dfs -du -s -h <ch>`	taille logique et espace brut consommé
`hdfs dfs -count [-q] <rép>`	répertoires, fichiers, octets (et quotas)
`hdfs dfs -stat "%n %b %o %r" <f>`	nom, taille, taille de bloc, réplication
`hdfs dfs -setrep [-w] <r> <ch>`	changer la réplication
`hdfs dfs -appendToFile <local> <f>`	ajouter à la fin d'un fichier HDFS
`hdfs dfs -getmerge <rép> <local>`	concaténer tout un répertoire vers un fichier local
`hdfs fsck <ch> -files -blocks -locations`	blocs, répliques et santé d'un chemin
`hdfs dfsadmin -report`	état des DataNodes et capacité
`hdfs dfsadmin -safemode get|enter|leave`	gérer le mode lecture seule
`hdfs dfsadmin -setQuota <n> <rép>`	limiter le nombre de fichiers et répertoires

➤ Exemple — lire une ligne de `-ls`

```
-rw-r--r--  2 hadoop supergroup  144395 2026-10-20 10:12
/user/etudiant/tp1/livres/carroll-alice.txt
```

Dans l'ordre : permissions, **facteur de réplication (2)**, propriétaire, groupe, **taille en octets**, date, chemin. Pour un répertoire, la réplication vaut « - ».

1.3 Pas à pas : cinq livres dans HDFS

Pas à pas 1.2 — Charger, explorer, vérifier

Les livres viennent du pack de données du cours (`ex1_livres/anglais_gutenberg`, domaine public). Placez-les dans le dossier `donnees/livres` : il est partagé avec le conteneur sous `/opt/donnees`.

1. Créer l'arborescence de travail et charger les livres :

Shell du cluster

```
$ hdfs dfs -mkdir -p /user/etudiant/tp1/livres /user/etudiant/tp1/archives
$ cd /opt/donnees/livres
$ hdfs dfs -put carroll-alice.txt shakespeare-hamlet.txt melville-moby_dick.
    txt \
      austen-emma.txt chesterton-brown.txt /user/etudiant/tp1/livres/
$ hdfs dfs -ls -h /user/etudiant/tp1/livres
Found 5 items
-rw-r--r--   2 hadoop supergroup    866.3 K ... /user/etudiant/tp1/livres/
    austen-emma.txt
-rw-r--r--   2 hadoop supergroup    141.0 K ... /user/etudiant/tp1/livres/
    carroll-alice.txt
-rw-r--r--   2 hadoop supergroup    397.1 K ... /user/etudiant/tp1/livres/
    chesterton-brown.txt
-rw-r--r--   2 hadoop supergroup      1.2 M ... /user/etudiant/tp1/livres/
    melville-moby_dick.txt
-rw-r--r--   2 hadoop supergroup    159.1 K ... /user/etudiant/tp1/livres/
    shakespeare-hamlet.txt
```

2 Lire sans télécharger :

Shell du cluster

```
$ hdfs dfs -head /user/etudiant/tp1/livres/carroll-alice.txt
[Alice's Adventures in Wonderland by Lewis Carroll 1865]

CHAPTER I. Down the Rabbit-Hole
...
$ hdfs dfs -cat /user/etudiant/tp1/livres/carroll-alice.txt | wc -l
3331
```

3 Voir les blocs d'un gros fichier avec une taille de bloc de 1 Mo, le minimum autorisé, pour éviter de manipuler des gigaoctets :

Shell du cluster

```
$ hdfs dfs -D dfs.blocksize=1048576 -put /opt/donnees/livres/bible-kjv.txt /
    user/etudiant/tp1/
$ hdfs dfs -stat "%n %b octets, bloc=%o, r=%r" /user/etudiant/tp1/bible-kjv.
    txt
bible-kjv.txt 4332554 octets, bloc=1048576, r=2
$ hdfs fsck /user/etudiant/tp1/bible-kjv.txt -files -blocks -locations
/user/etudiant/tp1/bible-kjv.txt 4332554 bytes, replicated: replication=2, 5
     block(s):  OK
0. BP-...:blk_1073741830_1006 len=1048576 Live_repl=2 [
    DatanodeInfoWithStorage[172.18.0.4:9866,...], ...]
...
4. BP-...:blk_1073741834_1010 len=138250 Live_repl=2 [...]
Status: HEALTHY
```

★ À retenir

`-D dfs.blocksize=...` et `-D dfs.replication=...` ne changent la valeur que **pour la commande en cours**. Le NameNode enregistre la taille de bloc et la réplication de **chaque fichier**.

✍ Attention ! — sous Windows natif

Les commandes sont identiques, mais `wc`, `head` et les barres obliques `\` de continuation de ligne n'existent pas dans `cmd`. Écrivez chaque commande sur une ligne, et utilisez PowerShell (`| Measure-Object -Line`) ou Git Bash.

Exercices du TP-1

1.1 — *Arborescence*

Créez en une seule commande `/user/etudiant/tp1/tmp/a/b`. Listez récursivement `/user/etudiant/tp1`.

1.2 — *Espace occupé*

Avec `-du -s -h`, donnez la taille logique du dossier `livres` (5 fichiers) et l'espace disque réellement consommé. Expliquez le rapport entre les deux nombres.

1.3 — *Calcul de blocs*

Sans machine : combien de blocs de 128 Mo pour un fichier de 1 Go ? De 129 Mo ? De 1 Ko ? Quel espace brut avec $r = 3$?

1.4 — *Le dernier bloc*

Vérifiez avec `fsck` que `bible-kjv.txt` (4 332 554 octets, blocs de 1 Mo) a 5 blocs, puis calculez la taille du dernier.

1.5 — *Copier, renommer, supprimer*

Copiez `carroll-alice.txt` dans `archives`, renommez la copie en `alice_2026.txt`, puis supprimez `tmp` et tout son contenu.

1.6 — *Réplication*

Passez la réplication de `austen-emma.txt` à 1, puis à 3. Que montre `fsck` avec 2 DataNodes ? Pourquoi ?

1.7 — *Fusion*

Récupérez les 5 livres dans **un seul** fichier local `tous.txt`. Quelle est sa taille ?

1.8 — *Ajout en fin de fichier*

Ajoutez la ligne « THE END » à la fin de `carroll-alice.txt` dans HDFS, sans le retélécharger. Vérifiez avec `-tail`.

1.9 — *Quota*

Placez un quota de **3 noms** sur un nouveau répertoire `tp1/quota`, puis essayez d'y copier 3 livres. Combien passent ? Pourquoi ?

1.10 — *Mode sécurisé*

Passez HDFS en *safe mode*, essayez de créer un répertoire, puis de lire un fichier. Concluez, puis quittez le mode sécurisé.

Corrigé express

1.1 `hdfs dfs -mkdir -p /user/etudiant/tp1/tmp/a/b` puis `hdfs dfs -ls -R /user/etudiant/tp1`.
1.2 `2.7 M  5.4 M` : 2 843 966 octets logiques ; brut = taille × réplication (2 sous Docker, 1 sous Windows).
1.3 1 Go → 8 blocs ; 129 Mo → 2 blocs (128 + 1) ; 1 Ko → 1 bloc, qui n'occupe que 1 Ko. Brut = 3 Go, 387 Mo et 3 Ko.
1.4 4 332 554 / 1 048 576 = 4,13 → 5 blocs ; dernier = 4 332 554 − 4 × 1 048 576 = **138 250** octets.
1.5 `-cp .../livres/carroll-alice.txt .../archives/` ; `-mv .../archives/carroll-alice.txt .../archives/alice_2026.txt` ; `-rm -r .../tp1/tmp`.
1.6 `hdfs dfs -setrep -w 1 ...` puis `-setrep 3 ...` (sans `-w`, qui attendrait indéfiniment). `fsck` : *Under-replicated blocks* — on ne place jamais 2 répliques sur le même DataNode : 2 DataNodes = 2 répliques au maximum.
1.7 `hdfs dfs -getmerge /user/etudiant/tp1/livres tous.txt` → 2 843 966 octets.
1.8 `echo "THE END" > fin.txt ; hdfs dfs -appendToFile fin.txt .../carroll-alice.txt ; hdfs dfs -tail ...`
1.9 `hdfs dfsadmin -setQuota 3 .../tp1/quota` : seuls **2** fichiers passent. Le répertoire compte lui-même pour 1 nom (`NSQuotaExceededException`). Vérifiez avec `hdfs dfs -count -q`.
1.10 `hdfs dfsadmin -safemode enter` : `mkdir` échoue (*Name node is in safe mode*), la lecture fonctionne. Le safe mode est un mode **lecture seule**. Puis `-safemode leave`.

2 YARN : ressources et applications

H. BADIR AI • Big Data — Écosystème Hadoop

YARN (*Yet Another Resource Negotiator*) est le « système d'exploitation » du cluster. Il partage la mémoire et le CPU entre les applications. Dans ce TP :

1. **Rappel : RM, NM, AM, containers**
2. **Les commandes `yarn` et `mapred`**
3. **Lancer, suivre et arrêter un job**
4. **Lire les logs, calculer les containers**
5. **Créer une file d'attente « dev »**

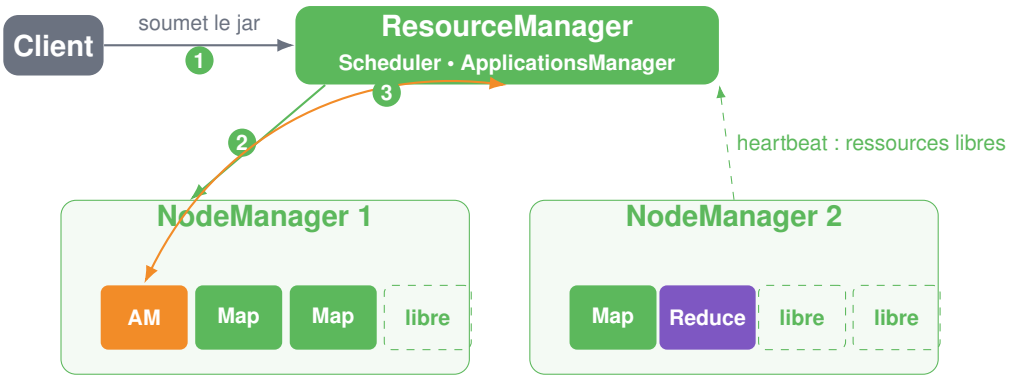
Le saviez-vous ?

Avant YARN (Hadoop 1), un seul démon, le JobTracker, gérait tout, avec une limite d'environ 4 000 nœuds. YARN (Hadoop 2, 2012) sépare la **gestion des ressources** du **suivi des jobs**. Spark, Tez ou Flink peuvent ainsi partager le même cluster.

2.1 Rappel : l'architecture de YARN

Définition — Les quatre acteurs

ResourceManager (RM) : un par cluster ; son *Scheduler* attribue les ressources. **NodeManager (NM)** : un par nœud ; il lance et surveille les containers. **ApplicationMaster (AM)** : un par application ; il négocie les containers et suit ses tâches. **Container** : une quantité de mémoire et de vcores sur un nœud précis.



1 soumission 2 le RM fait lancer l'AM 3 l'AM demande des containers et le RM les accorde puis l'AM fait lancer les tâches par les NM

Rappel 2.1 — Combien de containers ?

Un NodeManager offre `yarn.nodemanager.resource.memory-mb` de mémoire. Une tâche Map demande `mapreduce.map.memory.mb`. Donc :

$$\text{Containers par nœud} = \left\lfloor \frac{\text{mémoire offerte}}{\text{mémoire d'un container}} \right\rfloor$$

(et pas plus que les vcores)

Dans le cluster du TP-0 (Docker) : 2 NodeManagers $\times$ 4 096 Mo, et des containers de 1 024 Mo, soit **8 containers** en tout, dont 1 réservé à l'ApplicationMaster de chaque job.

2.2 Les commandes essentielles

Commande	Effet
`yarn node -list -all`	les NodeManagers et leur état
`yarn node -status <nodeId>`	mémoire et vcores d'un nœud
`yarn application -list [-appStates ALL]`	applications en cours (ou toutes)
`yarn application -status <appId>`	détail : état, file, progression, durée
`yarn application -kill <appId>`	arrêter une application
`yarn applicationattempt -list <appId>`	tentatives de l'ApplicationMaster
`yarn container -list <attemptId>`	containers d'une tentative
`yarn logs -applicationId <appId>`	logs agrégés (application terminée)
`yarn top`	vue « top » du cluster, rafraîchie en continu
`yarn queue -status <file>`	état d'une file d'attente
`yarn rmadmin -refreshQueues`	recharger les files sans redémarrer
`mapred job -list | -status <jobId>`	vue côté MapReduce

➤ Exemple — les identifiants

`application_1729420000000_0005` est l'application YARN (horodatage du démarrage du RM, puis n° 5). Le job MapReduce correspondant est `job_1729420000000_0005`, sa 1re tentative `appattempt_1729420000000_0005_000001`, et ses containers `container_1729420000000_0005_01_00000N` (le n° 1 est l'AM).

2.3 Pas à pas : lancer, suivre, arrêter

Pas à pas 2.2 — Le cycle de vie en direct

Ouvrez **deux** terminaux sur le cluster (`docker compose exec namenode bash`) : le premier lance les jobs, le second les observe.

1. Inspecter le cluster :

Terminal 2

```
$ yarn node -list -all
Total Nodes:2
         Node-Id      Node-State  Node-Http-Address  Number-of-Running-
    Containers
 a1b2c3d4e5f6:39427     RUNNING  a1b2c3d4e5f6:8042
       0
 f6e5d4c3b2a1:41231     RUNNING  f6e5d4c3b2a1:8042
       0
```

2. Lancer un job assez long (16 tâches Map), en arrière-plan :

Terminal 1

```
$ EX=$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.4.3.jar
$ hadoop jar $EX pi 16 1000000 > pi.log 2>&1 &
```

3. Le suivre :

Terminal 2

```
$ yarn application -list
Total number of applications (application-types: [], states: [SUBMITTED,
    ACCEPTED, RUNNING]):1
   Application-Id              Application-Name  Application-Type  User
    Queue    State    Final-State  Progress
application_1729420000000_0002  QuasiMonteCarlo   MAPREDUCE         hadoop
    default  RUNNING  UNDEFINED    35%
$ yarn application -status application_1729420000000_0002
$ yarn top                                  # q pour quitter
```

Sur `http://localhost:8088`, cliquez sur l'application, puis sur *ApplicationMaster* : on

voit les tâches Map se succéder par **vagues**.

4 Arrêter un job : relancez le même job, puis tuez-le :

Terminal 2

```
$ yarn application -kill application_1729420000000_0003
Killing application application_1729420000000_0003
$ yarn application -list -appStates KILLED
```

5 Lire les logs d'un job terminé, grâce à l'agrégation des logs activée au TP-0 :

Terminal 2

```
$ yarn logs -applicationId application_1729420000000_0002 > pi_logs.txt
$ grep -o "container_[0-9_]*" pi_logs.txt | sort -u | head -3
container_1729420000000_0002_01_000001
container_1729420000000_0002_01_000002
container_1729420000000_0002_01_000003
$ grep -o "container_[0-9_]*" pi_logs.txt | sort -u | wc -l
```

6 Un vrai traitement : le WordCount fourni par Hadoop, sur les 5 livres du TP-1.

Terminal 1

```
$ hadoop jar $EX wordcount /user/etudiant/tp1/livres /user/etudiant/tp2/wc
$ hdfs dfs -cat /user/etudiant/tp2/wc/part-r-00000 | sort -k2,2nr | head -3
the     25072
of      13789
and     13508
```

✍ Attention ! — le WordCount fourni par Hadoop

Il découpe le texte uniquement sur les **espaces** : « The », « the » et « the, » sont trois mots différents. Ses chiffres sont donc plus petits que ceux de notre propre WordCount du TP-3, qui met en minuscules et retire la ponctuation.

2.4 Pas à pas : une file d'attente « dev »

Pas à pas 2.3 — Capacity Scheduler

On partage le cluster entre la file `default` (70 %) et une file `dev` (30 %, jusqu'à 50 % si le cluster est libre).

1 **Docker** : dans le fichier `config`, remplacez les deux lignes `root.queues` et `root.default.capacity` par :

config (extrait)

```
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.queues=default,dev
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.default.capacity=70
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.dev.capacity=30
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.dev.maximum-capacity=50
```

puis recréez le ResourceManager : `docker compose up -d --force-recreate resourcemanager`.
Windows : ajoutez les mêmes propriétés dans `etc\hadoop\capacity-scheduler.xml`, puis lancez `yarn rmadmin -refreshQueues`.

2 Vérifier, puis soumettre un job dans la file `dev` :

Shell du cluster

```
$ yarn queue -status dev
Queue Information :
Queue Name : dev
        State : RUNNING
        Capacity : 30.0%
        Maximum Capacity : 50.0%
...
$ hadoop jar $EX pi -D mapreduce.job.queuename=dev 8 100000
```

Dans l'interface 8088, l'onglet *Scheduler* montre l'occupation de chaque file.

★ À retenir

Le **RM** décide *combien* et *où* ; l'**AM** décide *quoi* exécuter. Un job MapReduce = 1 container pour l'AM + 1 container par tâche Map ou Reduce. Les tâches qui ne trouvent pas de container libre attendent la **vague** suivante.

Exercices du TP-2

2.1 — *Ressources du cluster*

Quelle mémoire totale et combien de vcores YARN gère-t-il ? (interface 8088, ou `yarn node -status`).

2.2 — *Vagues*

Combien de containers de 1 Go peuvent tourner en même temps ? Pour `pi 16 1000000`, combien de tâches Map au maximum en parallèle, et combien de vagues ?

2.3 — *États*

Donnez la succession des états d'une application YARN qui réussit. Quelle différence entre

State et *Final-State* ?

2.4 — *Arrêt*

Lancez un job, tuez-le à 30 %. Quels *State* et *Final-State* affiche `-status` ? Les tâches déjà terminées sont-elles conservées ?

2.5 — *Logs*

Pour le premier job pi, combien de containers **différents** apparaissent dans `yarn logs` (dernière commande du pas à pas) ? Que représente le container n° 000001 ?

2.6 — *File inconnue*

Que se passe-t-il si l'on soumet un job dans la file `test`, qui n'existe pas ?

2.7 — *Container trop gros*

On lance un job avec `-D mapreduce.map.memory.mb=6000`. Que se passe-t-il, et pourquoi ?

2.8 — *Comparer*

Comparez `yarn application -list` et `mapred job -list` pendant un job : même numéro ?

Corrigé express

2.1 2 × 4 096 = **8 192 Mo** ; vcores : 2 × 8 = **16** (`yarn.nodemanager.resource.cpu-vcores` vaut 8 par défaut).
2.2 8 containers de 1 Go. 1 est pris par l'AM : **7 Map** au plus en même temps, donc 3 vagues pour 16 Map (7 + 7 + 2), puis le Reduce.
2.3 NEW → NEW_SAVING → SUBMITTED → ACCEPTED → RUNNING → FINISHED. *State* = cycle de vie YARN ; *Final-State* = verdict de l'application (SUCCEEDED, FAILED, KILLED, UNDEFINED tant qu'elle tourne).
2.4 State = KILLED, Final-State = KILLED. Aucun résultat n'est validé : pas de fichier `_SUCCESS` ni de `part-r-*` dans la sortie. Les sorties des Map terminées, stockées sur les disques locaux, sont supprimées.
2.5 1 AM + 16 Map + 1 Reduce = **18** containers, si aucune tâche n'a été relancée. Le n° 000001 est toujours l'ApplicationMaster.
2.6 Rejet immédiat : *submitted by user hadoop to unknown queue: test*.
2.7 L'AM (1 Go) démarre, mais aucun NodeManager n'offre 6 000 Mo (4 096 maximum) : les Map ne sont **jamais** allouées et le job reste bloqué à « map 0% ». Il faut le tuer.
2.8 Oui : `application_X_N` ↔ `job_X_N`. Un job MapReduce est une application YARN parmi d'autres.

3 MapReduce pas à pas : mots et amis communs

■ **H. BADIR AI** • Big Data — Écosystème Hadoop

MapReduce découpe un calcul en milliers de petites tâches indépendantes (**Map**), regroupe leurs résultats par clé (**Shuffle**), puis les agrège (**Reduce**). Deux cas réels, pas à pas :

- ➤ 1 **Rappel du modèle et préparation du projet**
- ➤ 2 **Cas 1 : compter les mots de 5 livres**
- ➤ 3 **Cas 2 : amis communs de 2 personnes**
- ➤ 4 **. . . puis de 3 et de 4 personnes**
- ➤ 5 **Amis entre eux ou non ? Passage à l'échelle**

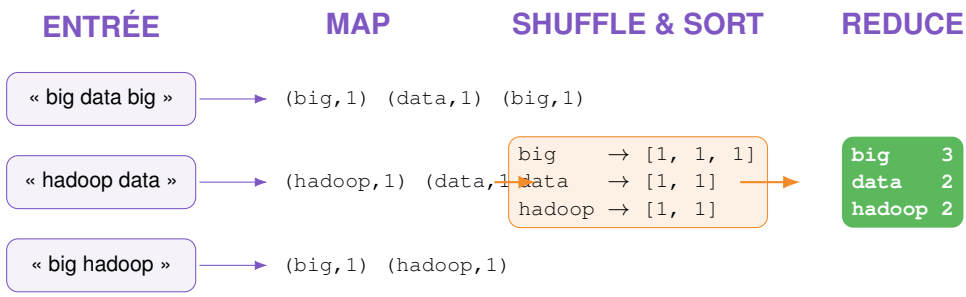
● Le saviez-vous ?

MapReduce a été publié par Google en 2004 (Dean et Ghemawat) pour construire l'index du Web. La fonctionnalité « amis en commun » des réseaux sociaux est un exemple classique de calcul MapReduce, souvent lancé chaque nuit.

3.1 Rappel : le modèle MapReduce

✦ Définition — Les deux fonctions

map : $(k_1, v_1) \to \text{liste}(k_2, v_2)$ — appelée une fois par enregistrement (une ligne).
reduce : $(k_2, \text{liste}(v_2)) \to \text{liste}(k_3, v_3)$ — appelée une fois par **clé distincte**, avec toutes ses valeurs.
Entre les deux, Hadoop **trie et regroupe** les paires par clé : c'est le **shuffle**.



★ À retenir

Résoudre un problème avec MapReduce, c'est d'abord **choisir la clé**. Tout ce qui doit être traité **ensemble** doit porter **la même clé**, car le shuffle le réunira dans le même appel à `reduce`.

3.2 Préparation : le projet Java

Pas à pas 3.1 — Un projet Maven pour les deux cas

1. Créez l'arborescence suivante dans le dossier `hadoop-docker` du TP-0 :

Terminal du PC

```
$ mkdir -p tp3/src/main/java/ma/bigdata/tp3/wordcount tp3/src/main/java/ma/
    bigdata/tp3/amis
```

2. Le fichier `tp3/pom.xml` :

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>ma.bigdata</groupId><artifactId>tp3</artifactId><version>1.0</
    version>
  <properties>
    <maven.compiler.release>8</maven.compiler.release>   <!-- s'exécute sur
    Java 8 ou 11 -->
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.apache.hadoop</groupId><artifactId>hadoop-client</
    artifactId>
      <version>3.4.3</version><scope>provided</scope>   <!-- fourni par le
    cluster -->
    </dependency>
  </dependencies>
  <build><finalName>tp3</finalName></build>
</project>
```

3 Compiler. Si Maven n'est pas installé, il peut tourner dans Docker. Déposez ensuite le jar dans `jobs/`, visible dans le cluster sous `/opt/jobs` :

Terminal du PC

```
$ cd tp3
$ docker run --rm -v "$PWD":/app -v "$HOME/.m2":/root/.m2 -w /app maven:3.9-
    eclipse-temurin-17 mvn -q package
$ cp target/tp3.jar ../jobs/
```

3.3 Cas 1 — Compter les mots dans 5 livres

Les 5 livres du TP-1 (*Alice*, *Hamlet*, *Moby Dick*, *Emma*, *Father Brown*) totalisent **55 654 lignes** et 2,8 Mo. Objectif : le nombre d'occurrences de chaque mot, toutes œuvres confondues.

1 Comprendre sans Hadoop

Un « pipe » Unix reproduit exactement MapReduce sur une seule machine :

Shell du cluster

```
$ cd /opt/donnees/livres
$ cat carroll-alice.txt shakespeare-hamlet.txt melville-moby_dick.txt austen-emma
    .txt chesterton-brown.txt \
    | tr -cs 'A-Za-z' '\n' | tr 'A-Z' 'a-z' | sort | uniq -c | sort -nr | head -3
  26940 the
  15283 and
  14119 of
```

Commande	Rôle MapReduce	Ce qu'elle fait
`tr -cs 'A-Za-z' '\n' | tr 'A-Z' 'a-z'`	Map	un mot par ligne, en minuscules
`sort`	Shuffle & Sort	rapproche les mots identiques
`uniq -c`	Reduce	compte chaque groupe

2 Concevoir

Conception 3.2 — Clé, valeur, combiner

Map : pour chaque mot m de la ligne $\rightarrow$ émettre $(m, 1)$. **Reduce** : $(m, [1, 1, \ldots]) \rightarrow (m, \sum)$.
Combiner : la somme est associative, donc le Reducer peut aussi servir de *Combiner*. Chaque livre envoie alors (the, 7 000) au lieu de 7 000 fois (the, 1).
Nettoyage : minuscules, découpage sur tout ce qui n'est pas une lettre (`[^\\p{L}]+`), et filtre optionnel de mots vides passé en option `-D`.

3 Le code complet

src/main/java/ma/bigdata/tp3/wordcount/WordCountLivres.java

```java
package ma.bigdata.tp3.wordcount;

import java.io.IOException;
import java.util.HashSet;
import java.util.Set;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.GenericOptionsParser;

/** Exercice 1 - Comptage des mots dans plusieurs livres. */
public class WordCountLivres {

  public static class MotMapper extends Mapper<LongWritable, Text, Text,
    IntWritable> {
    private static final IntWritable UN = new IntWritable(1);
    private final Text mot = new Text();
    private final Set<String> motsVides = new HashSet<>();
    private int longueurMin;

    @Override
    protected void setup(Context ctx) {
      Configuration conf = ctx.getConfiguration();
      longueurMin = conf.getInt("wc.longueur.min", 1);
      for (String m : conf.get("wc.mots.vides", "").split(",")) {
        if (!m.trim().isEmpty()) motsVides.add(m.trim().toLowerCase());
      }
    }

    @Override
    protected void map(LongWritable pos, Text ligne, Context ctx) throws
    IOException, InterruptedException {
      // tout ce qui n'est pas une lettre (\p{L}, accents compris) sert de sé
    parateur
      for (String m : ligne.toString().toLowerCase().split("[^\\p{L}]+")) {
        if (m.isEmpty()) continue;
        if (m.length() < longueurMin || motsVides.contains(m)) {
          ctx.getCounter("Mots", "IGNORES").increment(1);
          continue;
        }
        ctx.getCounter("Mots", "GARDES").increment(1);
        mot.set(m);
        ctx.write(mot, UN);
      }
    }
```

```
  }

  public static class SommeReducer extends Reducer<Text, IntWritable, Text,
    IntWritable> {
    private final IntWritable total = new IntWritable();

    @Override
    protected void reduce(Text mot, Iterable<IntWritable> uns, Context ctx)
    throws IOException, InterruptedException {
      int somme = 0;
      for (IntWritable u : uns) somme += u.get();
      total.set(somme);
      ctx.write(mot, total);
    }
  }

  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    String[] a = new GenericOptionsParser(conf, args).getRemainingArgs();   //
    accepte -D cle=valeur
    if (a.length != 2) {
      System.err.println("Usage : WordCountLivres [-D wc.mots.vides=le,la] <entré
    e> <sortie>");
      System.exit(2);
    }
    Job job = Job.getInstance(conf, "TP3 - comptage des mots");
    job.setJarByClass(WordCountLivres.class);
    job.setMapperClass(MotMapper.class);
    job.setCombinerClass(SommeReducer.class);
    job.setReducerClass(SommeReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(a[0]));
    FileOutputFormat.setOutputPath(job, new Path(a[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

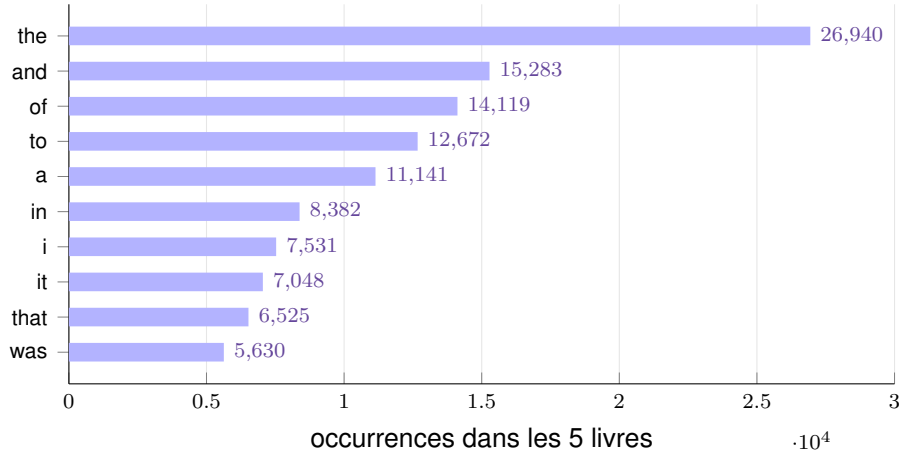
4 Exécuter sur le cluster

Shell du cluster

```
$ hadoop jar /opt/jobs/tp3.jar ma.bigdata.tp3.wordcount.WordCountLivres /user/
    etudiant/tp1/livres /user/etudiant/tp3/wc
...  map 100% reduce 100%
...  Job job_1729420000000_0007 completed successfully
        Map input records=55654
        Map output records=511244
        Combine input records=511244
        Combine output records=39051
        Reduce input groups=23892
        Reduce output records=23892
        Spilled Records=78102
$ hdfs dfs -cat /user/etudiant/tp3/wc/part-r-00000 | sort -k2,2nr | head -10
```

➤ Exemple — lire les compteurs

55 654 lignes lues → **511 244** mots émis (exactement le total du pipe Unix). Le Combiner les réduit à **39 051** paires (−92 %) avant le réseau. Il y a **23 892** mots distincts, donc 23 892 appels à `reduce`. 5 fichiers d'entrée donnent **5 tâches Map**, une par livre.



Top 10 sans filtre : uniquement des mots-outils.

5 Filtrer les mots vides

Shell du cluster

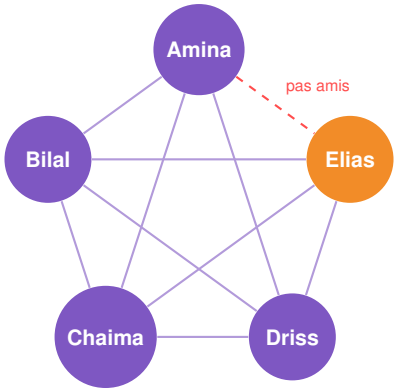
```
$ export LC_ALL=C.UTF-8
$ VIDES=the,of,and,to,a,in,i,it,that,was,he,you,his,her,as,with,for,had,is,my,on,
    at,be,not,but,she,me,so,by,all,him,they,this,have,which,said,there,were,we,
    them,their,what,or,will,when,are,no,one,if,would,an,been,upon,from,do,s,t,
    very,could,any,more,into,than,then,now,some,out,up,like,little,much,must,know
    ,only,see,well,how,should,can,did,about,never,who,such,think,good,go,man,
    thing,miss,mr,mrs,your,our,us,may,its,time,other,too,am,yet,over,own,these,
    those,shall,lord,thou,o,d,ll,here,come,old,where,before,down,after,great,way,
    just,say
$ hadoop jar /opt/jobs/tp3.jar ma.bigdata.tp3.wordcount.WordCountLivres \
    -D wc.longueur.min=3 -D wc.mots.vides=$VIDES /user/etudiant/tp1/livres /user/
    etudiant/tp3/wc2
$ hdfs dfs -cat /user/etudiant/tp3/wc2/part-r-00000 | sort -k2,2nr | head -5
whale   1228
emma    865
every   716
most    687
though  669
```

Les mots « propres » à un livre apparaissent enfin : *whale* pour Moby Dick, *emma* pour Emma. Le compteur `Mots/IGNORES` vaut 283 390 : plus de la moitié des mots sont des mots vides.

3.4 Cas 2 — Les amis communs (2, 3 puis 4 personnes)

Un petit réseau social de 5 personnes et 9 amitiés. **Amina et Elias ne sont pas amis.**

Chaque ligne du fichier donne une personne, puis la liste de ses amis.



donnees/amis.txt

```
Amina:Bilal,Chaima,Driss
Bilal:Amina,Chaima,Driss,Elias
Chaima:Amina,Bilal,Driss,Elias
Driss:Amina,Bilal,Chaima,Elias
Elias:Bilal,Chaima,Driss
```

Shell du cluster

```
$ hdfs dfs -mkdir -p /user/etudiant/tp3 && hdfs dfs -put /opt/donnees/amis.txt /
    user/etudiant/tp3/
```

3.4.1 Méthode 1 — deux amis : l'intersection des listes

Pas à pas 3.3 — Clé = la paire d'amis

1 **Map** : pour la ligne de P, pour chaque ami F, émettre la clé **(P, F) triée**, avec comme valeur la liste des amis de P.

Clé émise (triée)	Valeur (liste d'Amina)
`Amina,Bilal`	`Bilal,Chaima,Driss`
`Amina,Chaima`	`Bilal,Chaima,Driss`
`Amina,Driss`	`Bilal,Chaima,Driss`

2 **Shuffle** : la clé `Amina,Bilal` reçoit la liste d'Amina **et** celle de Bilal, qui émet la même clé triée.

3 **Reduce** : l'**intersection** des deux listes donne les amis communs.

AmisCommuns.java -- méthode 1

```
package ma.bigdata.tp3.amis;

import java.io.IOException;
import java.util.ArrayList;
```

```java
import java.util.Arrays;
import java.util.List;
import java.util.TreeSet;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

/** Exercice 2 - Amis en commun. Entrée : « personne:ami1,ami2,... » */
public class AmisCommuns {

  public static class PaireMapper extends Mapper<LongWritable, Text, Text, Text>
    {
    private final Text paire = new Text();
    private final Text liste = new Text();

    @Override
    protected void map(LongWritable pos, Text ligne, Context ctx) throws
    IOException, InterruptedException {
      String[] p = ligne.toString().split(":");
      if (p.length != 2) { ctx.getCounter("Amis", "LIGNES_INVALIDES").increment
    (1); return; }
      String personne = p[0].trim();
      liste.set(p[1].trim());                       // toute la liste d'amis de
    la personne
      for (String ami : p[1].split(",")) {
        ami = ami.trim();
        // clé triée : (Amina,Bilal) et (Bilal,Amina) deviennent la même clé
        String cle = personne.compareTo(ami) < 0 ? personne + "," + ami : ami + "
    ," + personne;
        paire.set(cle);
        ctx.write(paire, liste);
      }
    }
  }

  public static class IntersectionReducer extends Reducer<Text, Text, Text, Text>
     {
    private final Text communs = new Text();

    @Override
    protected void reduce(Text paire, Iterable<Text> listes, Context ctx) throws
    IOException, InterruptedException {
      List<TreeSet<String>> ensembles = new ArrayList<>();
      for (Text l : listes) ensembles.add(new TreeSet<>(Arrays.asList(l.toString
    ().split(","))));  // copie !
      if (ensembles.size() != 2) {                  // amitié déclarée d'un seul c
    ôté
        ctx.getCounter("Amis", "AMITIES_A_SENS_UNIQUE").increment(1);
        return;
      }
```

```
      TreeSet<String> inter = ensembles.get(0);
      inter.retainAll(ensembles.get(1));           // intersection des deux
    listes
      communs.set(inter.toString());
      ctx.write(paire, communs);
    }
  }

  public static void main(String[] args) throws Exception {
    Job job = Job.getInstance(new Configuration(), "TP3 - amis en commun");
    job.setJarByClass(AmisCommuns.class);
    job.setMapperClass(PaireMapper.class);
    job.setReducerClass(IntersectionReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

Shell du cluster

```
$ hadoop jar /opt/jobs/tp3.jar ma.bigdata.tp3.amis.AmisCommuns /user/etudiant/tp3
    /amis.txt /user/etudiant/tp3/m1
$ hdfs dfs -cat /user/etudiant/tp3/m1/part-r-*
Amina,Bilal     [Chaima, Driss]
Amina,Chaima    [Bilal, Driss]
Amina,Driss     [Bilal, Chaima]
Bilal,Chaima    [Amina, Driss, Elias]
Bilal,Driss     [Amina, Chaima, Elias]
Bilal,Elias     [Chaima, Driss]
Chaima,Driss    [Amina, Bilal, Elias]
Chaima,Elias    [Bilal, Driss]
Driss,Elias     [Bilal, Chaima]
```

✍ **Attention ! — la limite de la méthode 1**

Amina et Elias n'apparaissent pas, alors qu'ils ont 3 amis en commun (Bilal, Chaima, Driss) ! La méthode 1 ne fabrique de clés que pour des personnes **déjà amies**. Et elle ne se généralise pas simplement à 3 ou 4 personnes.

3.4.2 Méthode 2 — inverser le raisonnement (2, 3, 4… personnes)

Pas à pas 3.4 — « Je suis l'ami commun de tous les groupes de mes amis »

1. **Idée** : si Amina a pour amis {Bilal, Chaima, Driss}, alors Amina est une amie commune de **chaque groupe** de k personnes choisies parmi eux, **qu'ils soient amis entre eux ou non**.
2. **Map** : pour la ligne de P, énumérer tous les groupes de k amis (triés) et émettre **(groupe, P)**. Pour Amina avec $k = 2$:

Clé (groupe trié)	Valeur
`Bilal,Chaima`	`Amina`
`Bilal,Driss`	`Amina`
`Chaima,Driss`	`Amina`

3 **Reduce** : rassembler toutes les personnes reçues pour un groupe. Ce sont **ses amis communs**. Aucune intersection n'est nécessaire !

4 **Option** `-D amis.statut=true` : le graphe est copié sur chaque nœud (*cache distribué*), et le Reducer indique si les membres du groupe sont **tous amis entre eux** ou non.

AmisCommunsK.java -- méthode 2, groupes de k personnes

```
package ma.bigdata.tp3.amis;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.URI;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.TreeSet;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.GenericOptionsParser;

/**
 * Amis communs d'un groupe de k personnes (k = 2, 3 ou 4), qu'elles soient amies
    entre elles ou non.
 * Idée : si P a pour amis {a, b, c}, alors P est un ami commun de TOUS les
   groupes de k personnes
 * pris dans {a, b, c}. Le Mapper émet (groupe trié, P) ; le Reducer rassemble
   les P de chaque groupe.
 *   -D amis.k=3            taille des groupes (défaut 2)
 *   -D amis.min=2          n'afficher que les groupes ayant au moins 2 amis
   communs (défaut 1)
 *   -D amis.statut=true    indiquer si les membres du groupe sont tous amis
```

```
    entre eux (graphe en cache)
 */
public class AmisCommunsK {

  public static class GroupeMapper extends Mapper<LongWritable, Text, Text, Text>
     {
    private int k;
    private final Text groupe = new Text();
    private final Text personne = new Text();

    @Override
    protected void setup(Context ctx) { k = ctx.getConfiguration().getInt("amis.k
    ", 2); }

    @Override
    protected void map(LongWritable pos, Text ligne, Context ctx) throws
    IOException, InterruptedException {
      String[] p = ligne.toString().split(":");
      if (p.length != 2) { ctx.getCounter("Amis", "LIGNES_INVALIDES").increment
    (1); return; }
      personne.set(p[0].trim());
      String[] amis = new TreeSet<>(Arrays.asList(p[1].trim().split(","))).
    toArray(new String[0]);  // triés
      combinaisons(amis, k, 0, new ArrayList<String>(), ctx);
    }

    /** Énumère récursivement les sous-ensembles de taille k (déjà triés) et émet
     (groupe, personne). */
    private void combinaisons(String[] amis, int reste, int debut, List<String>
    courant, Context ctx)
        throws IOException, InterruptedException {
      if (reste == 0) {
        groupe.set(String.join(",", courant));
        ctx.write(groupe, personne);
        ctx.getCounter("Amis", "GROUPES_EMIS").increment(1);
        return;
      }
      for (int i = debut; i <= amis.length - reste; i++) {
        courant.add(amis[i]);
        combinaisons(amis, reste - 1, i + 1, courant, ctx);
        courant.remove(courant.size() - 1);
      }
    }
  }

  public static class CommunsReducer extends Reducer<Text, Text, Text, Text> {
    private int min;
    private boolean statut;
    private final Map<String, Set<String>> graphe = new HashMap<>();

    @Override
    protected void setup(Context ctx) throws IOException {
      min = ctx.getConfiguration().getInt("amis.min", 1);
      statut = ctx.getConfiguration().getBoolean("amis.statut", false);
      if (statut && new File("graphe").exists()) {        // copie locale du
    fichier d'entrée (cache distribué)
        try (BufferedReader br = new BufferedReader(new InputStreamReader(new
```

```
    FileInputStream("graphe"), StandardCharsets.UTF_8))) {
          String l;
          while ((l = br.readLine()) != null) {
            String[] p = l.split(":");
            if (p.length == 2) graphe.put(p[0].trim(), new HashSet<>(Arrays.
    asList(p[1].trim().split(","))));
          }
        }
      }
    }

    @Override
    protected void reduce(Text groupe, Iterable<Text> personnes, Context ctx)
    throws IOException, InterruptedException {
      TreeSet<String> communs = new TreeSet<>();
      for (Text p : personnes) communs.add(p.toString());
      if (communs.size() < min) return;
      String res = communs.size() + "\t" + communs;
      if (statut) res += "\t" + (tousAmis(groupe.toString().split(",")) ? "amis
    entre eux" : "PAS tous amis");
      ctx.write(groupe, new Text(res));
    }

    private boolean tousAmis(String[] g) {
      for (int i = 0; i < g.length; i++)
        for (int j = i + 1; j < g.length; j++) {
          Set<String> a = graphe.get(g[i]);
          if (a == null || !a.contains(g[j])) return false;
        }
      return true;
    }
  }

  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    String[] a = new GenericOptionsParser(conf, args).getRemainingArgs();
    Job job = Job.getInstance(conf, "TP3 - amis communs de groupes de " + conf.
    getInt("amis.k", 2));
    job.setJarByClass(AmisCommunsK.class);
    if (conf.getBoolean("amis.statut", false)) job.addCacheFile(new URI(a[0] + "#
    graphe"));
    job.setMapperClass(GroupeMapper.class);
    job.setReducerClass(CommunsReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);
    FileInputFormat.addInputPath(job, new Path(a[0]));
    FileOutputFormat.setOutputPath(job, new Path(a[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

Deux amis ($k = 2$)

Shell du cluster

```
$ J="hadoop jar /opt/jobs/tp3.jar ma.bigdata.tp3.amis.AmisCommunsK"
$ $J -D amis.k=2 -D amis.statut=true /user/etudiant/tp3/amis.txt /user/etudiant/
    tp3/k2
$ hdfs dfs -cat /user/etudiant/tp3/k2/part-r-*
Amina,Bilal    2  [Chaima, Driss]          amis entre eux
Amina,Chaima   2  [Bilal, Driss]           amis entre eux
Amina,Driss    2  [Bilal, Chaima]          amis entre eux
Amina,Elias    3  [Bilal, Chaima, Driss]   PAS tous amis
Bilal,Chaima   3  [Amina, Driss, Elias]    amis entre eux
Bilal,Driss    3  [Amina, Chaima, Elias]   amis entre eux
Bilal,Elias    2  [Chaima, Driss]          amis entre eux
Chaima,Driss   3  [Amina, Bilal, Elias]    amis entre eux
Chaima,Elias   2  [Bilal, Driss]           amis entre eux
Driss,Elias    2  [Bilal, Chaima]          amis entre eux
```

Les 10 paires possibles sont présentes : les 9 de la méthode 1, avec les mêmes résultats, **plus Amina–Elias**, signalée « PAS tous amis ».

Trois amis ($k = 3$)

Shell du cluster

```
$ $J -D amis.k=3 -D amis.statut=true /user/etudiant/tp3/amis.txt /user/etudiant/
    tp3/k3
Amina,Bilal,Chaima     1  [Driss]           amis entre eux
Amina,Bilal,Driss      1  [Chaima]          amis entre eux
Amina,Bilal,Elias      2  [Chaima, Driss]   PAS tous amis
Amina,Chaima,Driss     1  [Bilal]           amis entre eux
Amina,Chaima,Elias     2  [Bilal, Driss]    PAS tous amis
Amina,Driss,Elias      2  [Bilal, Chaima]   PAS tous amis
Bilal,Chaima,Driss     2  [Amina, Elias]    amis entre eux
Bilal,Chaima,Elias     1  [Driss]           amis entre eux
Bilal,Driss,Elias      1  [Chaima]          amis entre eux
Chaima,Driss,Elias     1  [Bilal]           amis entre eux
```

Quatre amis ($k = 4$)

Shell du cluster

```
$ $J -D amis.k=4 -D amis.statut=true /user/etudiant/tp3/amis.txt /user/etudiant/
    tp3/k4
Amina,Bilal,Chaima,Elias   1  [Driss]    PAS tous amis
Amina,Bilal,Driss,Elias    1  [Chaima]   PAS tous amis
Amina,Chaima,Driss,Elias   1  [Bilal]    PAS tous amis
```

➤ Exemple — vérifier à la main

Le groupe {Bilal, Chaima, Driss, Elias} n'apparaît pas : il faudrait une personne amie avec les quatre, et Amina n'est pas amie avec Elias. Les 3 groupes affichés contiennent Amina **et** Elias : ils ne peuvent pas être « tous amis ».

3.4.3 Passage à l'échelle : l'explosion combinatoire

Même job sur le réseau de **1 000 personnes** du pack de données (`amis_1000.txt`, environ 22 amis par personne), avec `-D amis.min=3` :

k	Paires émises par Map	Groupes distincts	Groupes avec ≥ 3 amis communs	Durée (local)
2	239 145	159 525	17 705	4 s
3	1 704 421	1 591 428	19 284	9 s
4	8 974 079	8 869 719	6 163	30 s

★ À retenir

Une personne qui a d amis émet $\binom{d}{k}$ paires : pour $d = 22$, cela fait 231 paires pour $k = 2$, 1 540 pour $k = 3$ et 7 315 pour $k = 4$. Le volume du shuffle croît très vite avec k. En production, on limite k, on filtre les personnes qui ont trop d'amis, ou on ne cherche les groupes que parmi des amis directs.

Exercices du TP-3

3.1 — *Combiner*

Relancez le Cas 1 en commentant `job.setCombinerClass(...)`. Comparez `Reduce shuffle bytes` et la durée. Le résultat change-t-il ?

3.2 — *Nombre de tâches*

Pourquoi le Cas 1 lance-t-il exactement 5 tâches Map ? Combien en lancerait-il pour les 18 livres du dossier `anglais_gutenberg` ?

3.3 — *Compter les paires émises*

Sans exécuter : combien de paires la méthode 2 émet-elle sur `amis.txt` pour $k = 2$, $k = 3$ et $k = 4$? Vérifiez avec le compteur `Amis/GROUPES_EMIS`.

3.4 — *Amitié à sens unique*

Ajoutez la ligne `Farid:Amina,Bilal`, sans modifier les autres lignes. Que donnent la méthode 1 et la méthode 2 ($k = 2$) pour la paire Amina–Bilal ?

3.5 — *Le plus grand groupe*

Sur `amis_1000.txt` avec $k = 3$, trouvez le groupe qui a le plus d'amis communs (`sort -k2,2nr`). Ses membres sont-ils amis entre eux ?

Corrigé express

3.1 Même résultat. Sans Combiner, les 511 244 paires traversent le shuffle au lieu de 39 051 : environ 4,7 Mo au lieu de 0,55 Mo (`Reduce shuffle bytes`), soit près de 9 fois plus, et un job plus lent.
3.2 Un split ne traverse jamais deux fichiers : 5 petits fichiers (< 128 Mo) donnent 5 splits, donc 5 Map. 18 livres $\rightarrow$ 18 Map.
3.3 $\sum_P \binom{d_P}{k}$ avec d = (3, 4, 4, 4, 3) : $k = 2 \rightarrow$ 3+6+6+6+3 = **24** ; $k = 3 \rightarrow$ 1+4+4+4+1 = **14** ; $k = 4 \rightarrow$ 0+1+1+1+0 = **3**.
3.4 Méthode 1 : toujours [Chaima, Driss] (compteur `AMITIES_A_SENS_UNIQUE` = 2 pour Amina–Farid et Bilal–Farid). Méthode 2 : Amina,Bilal $\rightarrow$ [Chaima, Driss, **Farid**] — Farid se déclare ami des deux, il est compté.
3.5 `Driss00723,Meryem00747,Nabil00712` $\rightarrow$ 8 amis communs, « PAS tous amis ».

4 Références

1. APACHE SOFTWARE FOUNDATION. *Hadoop: Setting up a Single Node Cluster*, documentation 3.4.x. hadoop.apache.org.
2. APACHE SOFTWARE FOUNDATION. *HDFS Commands Guide* et *YARN Commands*, documentation 3.4.x.
3. APACHE SOFTWARE FOUNDATION. *MapReduce Tutorial* (WordCount v1.0 et v2.0).
4. DEAN, J. ; GHEMAWAT, S. *MapReduce: Simplified Data Processing on Large Clusters*. OSDI, 2004.
5. VAVILAPALLI, V. K. et al. *Apache Hadoop YARN: Yet Another Resource Negotiator*. SoCC, 2013.
6. WHITE, T. *Hadoop: The Definitive Guide*. 4e éd. O'Reilly, 2015.
7. Image Docker officielle `apache/hadoop` (Docker Hub) ; binaires `winutils` : dépôt GitHub `kontext-tech/winutils`.
8. Données : corpus Gutenberg de NLTK (domaine public) ; réseaux d'amis synthétiques du pack de données du cours.

« Sans big data, vous êtes aveugle et sourd, au milieu d'une autoroute. »
— Geoffrey Moore (attribué)

Doc. Hadoop 3.4.3

Image apache/hadoop